

Coloring AI - AI Engineering & Research Prompts

v0.2 | Productionization, sensing, experiment design, validation and export

14 August 2026

How to use these prompts

These prompts are designed for strong coding/research agents working on the Coloring AI / VitalSignAI physiological-personalization MVP. They are intentionally specific enough to produce reviewable engineering output rather than generic suggestions.

Global rules for every agent:

- Prefer current, maintained, primary-source technologies. Do not choose `face-api.js` / `MobileNetSSD` as the default greenfield production stack when `MediaPipe Tasks` and the dedicated `rPPG` stack cover the requirement more directly.
- Browser stack: `React + TypeScript + Vite + PWA` unless a concrete technical reason justifies a different choice.
- Camera: `navigator.mediaDevices.getUserMedia()` with explicit permission UX and mobile Safari handling.
- Primary physiological engine: `Vital Camera SDK` / `FacePhys`, pinned to a reviewed commit or released version and self-hosted where feasible.
- Face/behavior: `MediaPipe Face Landmarker`. Body/respiration motion: `MediaPipe Pose Landmarker`.
- Raw camera frames and biometric inference stay on-device by default.
- Do not claim medical-grade accuracy until the exact production build is benchmarked against synchronized contact sensors.
- Call camera-derived beat-interval variability `PRV` / pulse variability, not `ECG HRV`.
- Do not infer stress from heart rate alone.
- Do not describe facial-expression probabilities as direct access to internal emotion.
- The display-color/rPPG illumination confound is a launch-blocking validation item, not an edge case.
- Return tests, exact dependency versions/commits, assumptions, failure modes, performance measurements and privacy boundaries.

Prompt 1 - Productionize the full browser application

You are a senior full-stack engineer specializing in browser ML, privacy-preserving computer vision, TypeScript and physiological sensing.

Start from the provided Coloring AI static MVP and refactor it into a production-grade `React + TypeScript + Vite PWA` for mobile and desktop.

Requirements:

1. Integrate `KegangWangCCNU/vitalcamera-sdk` as the primary physiological sensor using a pinned reviewed version/commit. Do not follow GitHub `main` in production.
2. Self-host model files, WASM/runtime dependencies and static assets where licensing allows.
3. Add `MediaPipe Face Landmarker` using `@mediapipe/tasks-vision` for face landmarks, blendshapes/expression features and transformation matrices where useful.
4. Add `MediaPipe Pose Landmarker` for upper-torso/shoulder motion needed by respiration research.
5. Keep raw video and inference local. No frame upload. Derived metrics may only leave the device after explicit informed opt-in.
6. Implement a typed `SensorAdapter` interface so the `rPPG` engine can be replaced or compared later.
7. Implement `VitalCameraAdapter`, `DemoAdapter`, and a stub `ConsensusAdapter` for a future second estimator.

8. Use Web Workers for expensive synchronous MediaPipe or signal-processing work where needed so the main UI remains responsive.
9. Implement robust permission, no-camera, insecure-context, unsupported-browser, low-light, excessive-motion and model-load failure states.
10. Optimize for iOS Safari and Android Chrome as first-class targets, not afterthoughts.
11. Add a quality gate before experiments begin and pause/reject trials if signal quality drops below configured thresholds.
12. Add PWA manifest, offline shell, installability, responsive layout, accessibility and reduced-motion support.
13. Add unit tests for randomization/scoring/signal gates and Playwright end-to-end tests for the complete consumer flow.
14. Do not make medical claims.
15. Provide a dependency/license inventory and explain the Vital Camera Privacy Protection Addendum implications.

Deliver:

- PR-sized implementation.
- Architecture diagram.
- Technical README.
- Browser/device matrix.
- Performance profile: FPS, CPU, memory, battery/thermal observations.
- Exact dependency versions and reviewed commits.
- Known limitations and launch blockers.

Prompt 2 - Build the sensor fusion and consensus layer

You are an rPPG systems engineer. Build a modular local **SensorHub** that unifies physiology, motion, affect and quality into one timestamped stream.

Primary inputs:

- Vital Camera / FacePhys: BVP, HR, beat intervals, SQI, RMSSD/SDNN side outputs if available.
- MediaPipe Face Landmarker: landmarks, blendshapes, head pose / transformation signals.
- MediaPipe Pose Landmarker: shoulder/chest/torso motion.
- Optional second rPPG estimator: POS, CHROM or PRISM-style implementation.

Requirements:

- Normalize timestamps to a single monotonic clock.
- Expose typed events for `bvp`, `heart_rate`, `beat`, `prv`, `respiration`, `expression`, `gaze`, `head_pose`, `torso_motion`, `quality`, `stimulus` and `trial_state`.
- Track estimator disagreement and never silently average incompatible/low-quality signals.
- Build confidence from SQI, motion, illumination stability, face visibility, valid-beat count and repeat consistency.
- Provide configurable rules for rejecting impossible or unstable values.
- Keep all raw camera data local.
- Write deterministic simulation tests where one estimator is corrupted, one has delay, and one has screen-light artifacts.

Deliver code, tests, a data dictionary and a compact session JSON schema.

Prompt 3 - Build a rigorous randomized experiment engine

Act as a psychophysiology research engineer and statistician. Replace the simple randomized block runner with a reproducible within-subject experiment engine.

Requirements:

- Seeded randomization recorded in the session log.
- Balanced/counterbalanced ordering rather than naive shuffle when sample structure allows it.
- Two repeated blocks per condition in quick mode.
- Baseline and neutral washout periods.

- Predeclared transition-artifact exclusion window.
- Timestamped stimulus event log at millisecond precision.
- Trial-level SQI and motion gating.
- Minimum valid-second and valid-beat thresholds.
- Separate stages for color, font and emoji so effects are not confounded.
- Optional finalist stage: top two colors x top two fonts, tested only after the independent main-effect stages.
- Optional successive-halving / adaptive confirmation that allocates more time to close finalists rather than lengthening every condition.
- Bootstrap confidence intervals for within-user rankings.
- Optional self-report only after finalists to limit demand characteristics.
- Export suitable for mixed-effects and hierarchical analysis across users.

Primary quick-mode endpoint should be a transparent composite calming score. It must remain a research/product heuristic until validated.

Deliver:

- Experiment state machine.
- Randomization implementation.
- JSON schema.
- Synthetic-effect simulations proving ranking recovery.
- Tests for missing/low-quality windows and ties.
- A one-page protocol summary for researchers.

Prompt 4 - Solve the screen-color / rPPG illumination confound

You are a remote-photoplethysmography researcher. The application deliberately changes screen hue while measuring rPPG with the front camera. The display therefore changes the illumination spectrum on the face and may create a false physiological effect.

Design and implement a mitigation and validation program.

Requirements:

1. Define color stimuli in a perceptual color space such as OKLCH and match perceptual lightness as closely as practicable.
2. Keep a constant neutral region near the camera and/or around the sensor ROI where product design permits.
3. Log exact stimulus color, screen brightness when accessible, camera-frame RGB/luminance, face ROI RGB, SQI, pose, exposure-related metadata if exposed, and device/browser information.
4. Discard transient windows after visual changes.
5. Insert neutral washout periods.
6. Implement a non-pulsatile/global illumination reference channel and test whether it improves BVP stability.
7. Compare FacePhys against at least one physically different estimator such as POS/CHROM/PRISM.
8. Build an internal synchronized validation experiment using a contact PPG reference while the screen changes colors but the participant remains physiologically stable.
9. Quantify apparent camera HR/BVP shift caused by each stimulus/device combination.
10. Produce a go/no-go threshold for enabling any user-facing “calming color” claim.

Deliver code, plots, reproducible protocol, artifact metrics, correction experiments and a launch recommendation.

Prompt 5 - Add respiration robustly using two independent channels

Act as a respiratory-signal-processing engineer.

Add browser-side respiration estimation with two independent channels:

- A. Respiratory modulation extracted from the rPPG/BVP waveform.
- B. Upper-torso/shoulder motion from MediaPipe Pose Landmarker or a lightweight optical-flow/ROI method.

Requirements:

- Output `respiration_rate`, per-channel confidence, fused confidence and rejection reason.
- Fuse only when channels agree within a configurable tolerance or when one channel is demonstrably high confidence and the other is unavailable.
- Detect talking, large body movement, camera movement and poor framing.
- Output a `motion_respiration_amplitude_proxy` separately.
- Do not call the amplitude proxy “breath depth” until validated against a respiratory belt or equivalent reference.
- Benchmark slow breathing 4-8/min, ordinary resting breathing, faster breathing, talking and mild movement.
- Measure latency and the minimum window length needed for a stable estimate.

Deliver implementation, tests and reference-validation report.

Prompt 6 - Build calibrated affect, motion and Emoji Mirror modules

Act as an affective-computing researcher and browser ML engineer.

Build an affect/behavior layer that does not overclaim emotion recognition.

Use MediaPipe Face Landmarker blendshapes/landmarks as the always-available browser layer. Benchmark EmotiEFLib/ONNX as an optional higher-level expression model if it can meet the privacy/performance target.

Requirements:

- Maintain a per-session neutral-expression calibration.
- Prefer continuous features such as positive/negative expression tendency, arousal proxy, smile probability, mouth/blink/gaze/head-motion measures over a single categorical “emotion” label.
- Weight facial-expression outputs lower than physiology in stress-like classification.
- Add a separate Emoji Mirror stage using a predefined set such as warm smile, bright smile, relieved/calm, neutral and tense.
- Keep background, font, brightness and chat content fixed during Emoji Mirror trials.
- Never tell users to imitate the emoji.
- Measure spontaneous changes in smile-related blendshapes/landmarks, blink, gaze, head motion, HR and respiration.
- Return “strongest spontaneous mimicry response” rather than “this emoji makes you happy.”

Deliver calibration logic, experiment code, validation plan and result visualizations.

Prompt 7 - Build the consumer UI and live “wow” experience

You are a world-class consumer product designer and frontend engineer. Design the first 90 seconds of Coloring AI so users understand the benefit, trust the privacy model and are motivated to finish the experiment.

Requirements:

- Landing promise: find the colors that are associated with your calmest response - not a medical diagnosis and not a universal color claim.
- Explain camera-only local sensing in plain language.
- Show a compact real-time sensor card after consent: heart rate, breathing estimate, signal quality and one or two motion/expression signals.
- Avoid a dense medical dashboard.
- Add a camera-quality coach: face position, lighting, motion and talking.
- Use a familiar chat interface for trials.
- Keep chat copy emotionally neutral and comparable across trials.
- Change visual variables via CSS custom properties and a single stimulus controller.
- Provide clear progress, pause, resume and restart states.
- Make the results highly shareable: calm palette, activating palette, confidence, observed deltas, wallpaper preview and export actions.
- Follow WCAG contrast rules for all resulting bubble/text combinations.
- Do not expose raw biometric probabilities that could confuse users.

Deliver Figma-quality UI specification plus production React components and responsive states.

Prompt 8 - Build WhatsApp, Telegram and cross-app export

Act as a mobile-product engineer.

Build an export layer that converts the individualized visual result into portable assets and instructions.

Requirements:

- Generate portrait and landscape PNG wallpapers locally.
- Produce exact HEX and OKLCH values for background, incoming bubble, outgoing bubble, accent and text.
- Automatically choose accessible bubble/text combinations with measurable contrast.
- Provide device-aware dimensions and a “Copy color recipe” action.
- For WhatsApp, use only customization capabilities confirmed in current official Help Center documentation. Wallpaper/chat color are safe primary outputs. Do not promise arbitrary font-family control for ordinary WhatsApp when it is not available to that user/platform.
- If a WhatsApp account/platform exposes additional chat-font controls, detect/explain them conditionally rather than assuming them.
- For Telegram, investigate current official custom-theme, accent and wallpaper capabilities and generate the most portable supported artifact or instructions.
- Add generic exports for other apps: wallpaper PNG, palette JSON, CSS variables and design-token JSON.

Deliver implementation, platform capability matrix and current official source links.

Prompt 9 - Benchmark the entire sensing stack

Act as an rPPG benchmark lead. Create a reproducible benchmark comparing the primary and secondary candidates.

Include where code/licensing permits:

- Vital Camera SDK / FacePhys.
- open-rppg FacePhys.
- PhysMamba.
- RhythmMamba.
- CanonicalPhys-enhanced approaches.
- PRISM.
- POS and CHROM baselines.
- Reperio-rPPG and any newer 2026 public model with reproducible code that materially changes the comparison.

Evaluate:

- HR MAE, RMSE, correlation and failure/rejection rate.
- BVP signal quality / SNR where comparable.
- Cross-dataset generalization.
- Motion and head-pose robustness.
- Dynamic-illumination robustness.
- Skin-tone subgroup performance.
- Model/runtime size.
- Browser feasibility.
- CPU/GPU/WASM latency and memory.
- Mobile thermal/battery behavior.

Do not rank incompatible benchmark MAEs as if they were directly comparable.

Then run the same candidates on an internal webcam set synchronized with contact PPG. Recommend a production primary, a lightweight fallback and a disagreement/consensus strategy.

Prompt 10 - Mobile performance, reliability and privacy hardening

You are a staff web-platform engineer. Make the application dependable across real consumer devices.

Requirements:

- Test recent iPhone/Safari, Android/Chrome, macOS Chrome/Safari and Windows Chrome/Edge.

- Build camera constraints/fallbacks for front camera, frame rate, resolution and orientation changes.
- Handle browser tab backgrounding, screen lock, camera revocation, incoming calls and device rotation.
- Move heavy vision work off the main thread where possible.
- Create adaptive quality/performance modes for weak phones.
- Monitor dropped frames and thermal throttling locally without transmitting biometric data.
- Add CSP, SRI/self-hosting strategy, dependency pinning and supply-chain review.
- Verify no accidental video frames, canvas snapshots, BVP samples or derived biometrics are sent by analytics/error-reporting SDKs.
- Add privacy-preserving error telemetry containing only technical events unless the user separately opts into research data sharing.

Deliver test matrix, performance budget, privacy data-flow audit and release checklist.

Prompt 11 - Statistical analysis and personalization research plan

You are a psychophysiology statistician. Design the analysis for both single-user personalization and population-level learning.

Requirements:

- Primary within-user outcome: preregistered composite calming score.
- Report each component separately so the composite cannot hide contradictions.
- Estimate repeated-condition reliability and rank stability.
- Use bootstrap intervals for individual winner/runner-up margins.
- Define when results are “uncertain” instead of forcing a winner.
- Across users, use mixed-effects/hierarchical models with participant random effects and fixed effects for color/font/emoji, order, device, brightness, lighting and session time.
- Test interaction effects only after main effects have adequate power.
- Predefine handling of low SQI, missing beats and rejected trials.
- Investigate whether self-reported calm aligns with physiological response without using self-report to retroactively tune the primary endpoint.
- Separate exploratory analyses from confirmatory claims.

Deliver an analysis plan, data dictionary, example R/Python notebook and preregistration-ready methods section.

Primary technical sources to verify during implementation

- Vital Camera SDK: <https://github.com/KegangWangCCNU/vitalcamera-sdk>
- FacePhys Demo: <https://github.com/KegangWangCCNU/FacePhys-Demo>
- FacePhys paper: <https://arxiv.org/abs/2512.06275>
- open-rppg: <https://github.com/KegangWangCCNU/open-rppg>
- MediaPipe Face Landmarker for Web: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/web_js
- MediaPipe Pose Landmarker for Web: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker/web_js
- CanonicalPhys code: <https://github.com/infracore/CanonicalPhys>
- CanonicalPhys paper: <https://arxiv.org/abs/2607.15995>
- EmotiEffLib: <https://github.com/sb-ai-lab/EmotiEffLib>
- Reperio-rPPG: <https://github.com/deconasser/Reperio-rPPG>
- WhatsApp chat theme / color / wallpaper: <https://faq.whatsapp.com/6298875896807790>
- Telegram themes: <https://core.telegram.org/themes>